

Vendor Estimation Matrix Sharing, Implementation Plan

Status (dev, 2026-05-29; nothing on prod): - **M-01 (Phase 0) DONE + committed** (`7af8074`), migration `20260529000001`, `resolve_owner` shim, owner-aware `matrix.py / workflow_steps.py`. Verified on dev (`kwrlqqnzcnpjqvesygxo`). - **M-02 / Phase 1 (M1) DONE, applied to dev** (backend `899501f`, frontend+test `201ff60`), migration `20260529000002` (`link_id + em_link_vendor_only` CHECK + 4-col `uq_em_owner` NULLS NOT DISTINCT); `lib/estimate/effective.py` resolver; `require_vendor_link` in `lib/handshake.py`; `matrix.py` per-link override write + `?linkId=` effective read + 4-col `on_conflict`; **vendor override UI** in OrgHub `EstimatesTab` (studio-link selector → per-link effective matrix, accent-highlighted override cells); unit test `scripts/test_effective_matrix.py`. Validated: `on_conflict` binds the NND index (§6.1 closed, no fallback); CHECK rejects studio+link rows (§6.6); overlay logic (6 assertions); frontend builds. - Dev is **Postgres 17** → §6.2 closed for dev. - **Phase 2 (M2) BACKEND DONE, applied to dev** (commit `82c4a05`), migration `20260529000003` (3 share tables + `create_estimate_share` RPC for atomic find-or-create-series + supersede + insert + logging); `lib/estimate/projector.py` (granularity exposure boundary), `lib/estimate/delivery.py` (`route_inbox` strategy), `routes/estimate_share.py` (vendor targets/create/outbox/revoke + studio inbox/view), registered in `main.py`. Validated on dev: RPC end-to-end (1 series, supersede → one-live, `shared ×2/ superseded ×1` logging, cleaned up); projector exposure test (§6.5); effective + projector unit tests pass; router imports. - **M3 (Phase 2 frontend) DONE:** `ShareEstimatesModal` (granularity radio + expiry/label + live preview via new `GET /api/estimate-shares/preview` + unset-cell warning, §6.7) and an outbox section with View/Revoke wired into `StudioConnections.jsx`. Shared read-only `EstimateSnapshotView` component renders a snapshot by granularity (expandable breakdown). - **M4 (Phase 3) DONE:** studio "Received estimates" inbox section in `VendorConnections.jsx` (`GET /inbox`, read-only `EstimateSnapshotView`, best-effort `POST /{id}/view` on expand). - **Backend addition:** `GET /api/estimate-shares/preview?link_id=&granularity=` (read-only project, returns `{snapshot, unset_cells, profile_count}`); `create_share` refactored onto a shared `_build_snapshot` helper. Frontend builds clean; projector + effective-matrix tests pass. - **Phase 0 completeness fix:** `routes/fields.py` (`/fields`, `/field-values`, `/asset-combinations`, the Setup wizard's variable/value discovery) was still `require_studio` + hardcoded `owner_type='studio'`, so a vendor hit "Studio access required" on Setup. Now owner-aware (`_owner_scope` resolves `owner_type` from role; queries the org's own `source_field_mappings / replicated_assets`). Vendors derive estimation variables from their own synced source data (both dev test vendors have it). No manual-variable path needed. - **SMOKE TEST PASSED (2026-05-29):** full vendor-login chain working: vendor authors base matrix + per-link overrides, shares at a granularity, studio receives in inbox. Feature complete for v1. API: `/api/estimate-shares` (`GET /targets`, `GET /preview`, `POST ""`, `GET /outbox`, `POST /{id}/revoke`, `GET /inbox`, `POST /{id}/view`). Still nothing on prod.

Summary: Generalize the studio-only estimation stack to vendors, let a vendor maintain per-studio-link rate variations on top of a base matrix, and let the vendor share a frozen, granularity-controlled

snapshot of those estimates with a linked studio. v1 is visible-only on the studio side, no scenario-planner consumption.

Revision note (R2): resolves the two pre-M-01 core questions and three smaller items from design review, (1) the `owner_key` arbiter is now fully specified as a single enforcer (§2.2), not a half-state hybrid; (2) the RLS-vs-service-role contradiction is resolved explicitly: the route-layer owner filter is the enforced boundary, RLS is defense-in-depth, and M-01 must extend the existing studio-only estimate-table policies (§2.4, §3.2, §6.4); (3) auto-supersede on re-share (§2.6/§4.4), a `link_id` CHECK (§3.3/§6.6), and the verified-safe `estimate_config` PK swap (§3.2).

Revision note (R3): two model corrections. (1) **Replace semantics**, the series channel is keyed on (`vendor_id`, `link_id`) only; granularity is a per-share dispatch property, not part of the channel key. Keying on granularity was an IP footgun: a vendor dialing disclosure *down* (`workflow_step` → `asset_total`) would land in a different channel and leave the detailed share live, the §6.5 leak, one layer up. Now any re-share supersedes the prior regardless of granularity, and §2.4's modal framing is truthful (§2.4, §2.6, §3.4, §3.5, §4.4, §6.5). (2) **At-most-one-live is structural**, a partial unique index on `estimate_share_dispatches` (`series_id`) where `superseded_at` is null and `revoked_at` is null replaces the race-prone route-only invariant (§3.5, §4.4, §6.8), consistent with the §2.1 DB-integrity preference.

1. Findings

1.1 The estimation stack is studio-only, end to end

`workflow_steps`, `workflow_step_dependencies`, `estimate_config`, and `estimate_matrix` (20260501000003_workflow_steps.sql) are all keyed on `studio_id` NOT NULL → `studios(id)`. Every handler in `routes/matrix.py` and `routes/workflow_steps.py` is gated behind `require_studio` and reads `user.studio_id` directly. **Vendors have no matrix today.** [OrgHub.jsx](#) renders the Estimates/Workflows tabs unconditionally (`TABS = ['Members', 'Estimates', 'Workflows', 'Settings']`), so a vendor user can open them, but the API calls 403. Lighting up vendor authoring is purely a backend-scope change; the frontend surface already exists.

1.2 Matrix granularity and the craft axis

`estimate_matrix` is one row per (`studio_id`, `workflow_step_id`, `variable_values`) → `estimate_days` `numeric(10,2)`, `unique(studio_id, workflow_step_id, variable_values)`. The "Default" column is the row with `variable_values = {}`. `variable_values` is JSONB and is usable in the existing unique constraint (jsonb has a btree opclass).

`workflow_steps.craft` (nullable text) is the **only** grouping key available for the `craft_bucket` projection, the projector groups steps by `craft` and sums within each group.

`estimate_config.variable_fields` `text[]` defines the profile axes (the columns).

1.3 Auth already carries both org identities

`lib/auth.py`: `CurrentUser` carries **both** `studio_id` and `vendor_id` (one is null), plus `role ∈ {studio, vendor}`. `require_studio` / `require_vendor` exist; there is **no generic owner-resolution helper yet**. Membership is cached 15s. This means owner-aware routes need only a small `resolve_owner(user) → (owner_col, owner_id)` shim, not an auth rewrite.

1.4 The payload pattern is already inbox/route-scoped, token was removed

`routes/payload.py` `dispatch_payload` builds a **frozen** `payload_data` JSONB snapshot (`{asset_global_id, schema, data, dispatched_at}`), sets `expires_at`, inserts a dispatch row scoped to `recipient_vendor_id`, and writes an append-only `payload_access_log` row via `_log(...)`. Revoke sets `revoked_at`; `_assert_valid` raises **410** on revoked/expired. The one-time plaintext token was **dropped** in `20260503000004_remove_payload_token.sql`, "All vendor access goes through the authenticated vendor-inbox endpoint." Handshake gating uses `require_active_link(studio_id, vendor_id)` from `lib/handshake`. All reads/writes go through the **service-role client** in `lib/db.py`; the inbox is **route-scoped**, not RLS-enforced at runtime (see §1.7, §2.4).

Consequence: the delivery decision (route-scoped inbox; keep snapshot/revoke/expiry/log; no token) is a faithful reuse of the *current* payload pattern, not a new mechanism. `payload_dispatches` still FKs `asset_id → canonical_assets NOT NULL`, the one structural thing the estimate share deliberately omits (§2.5).

1.5 The handshake link

`20260507000002_studio_vendor_handshake.sql`: `studio_vendor_links` has one active row per (`studio_id`, `vendor_id`) pair (`studio_vendor_links_active_unique` where `status='active'`), RLS-readable by both parties (`svl_studio`, `svl_vendor`). This is the natural anchor for "1 matrix per studio handshake" and the scoping key for a share's recipient.

1.6 Existing matrix consumers (must stay studio-scoped)

`lib/scenario/shared.py` `fetch_validation_data(studio_id)` and `lib/scenario/context.py` read `workflow_steps` / `estimate_config` / `estimate_matrix` filtered by `studio_id=eq.{studio_id}`. Under the dual-FK generalization (§2.1), studio rows keep `studio_id` populated and vendor rows have it null, so these filters keep returning exactly the studio's rows **unchanged**, no scenario code change, and no risk of a vendor matrix leaking into studio scenario planning. This is the main reason dual-FK is preferred over an `owner_type/owner_id` rewrite (§2.1).

1.7 RLS is already present on the estimate tables, and is bypassed by the routes

`20260508000002_rls_expand.sql` enabled RLS and added **studio-only, JWT-claim** policies to `workflow_steps` (`ws_studio`), `workflow_step_dependencies` (`wsd_studio`, FK-chain through steps), `estimate_config` (`ec_studio`), and `estimate_matrix` (`em_studio`), each `... = (auth.jwt() -> 'app_metadata' ->> 'studio_id')::uuid`. The migration's own smoke test states the posture plainly: *studio JWT → own rows only; service role → all rows on all tables*. **Because every route uses the service-role client (§1.4)**, RLS is bypassed at runtime **and the policies only fire for a user-JWT**

PostgREST client (which the app does not use, the frontend goes through FastAPI). The payload tables follow the same model: `payload_access_log` has SELECT policies for both parties but no INSERT policy, "all writes go through the service-role `_log()` helper."**

Two consequences for this feature: - The runtime isolation boundary is the **route-layer owner filter**, not RLS (§2.4, §6.4). - Generalizing the estimate tables to vendors means the existing **studio-only** policies must be **extended to cover vendor rows** (M-01), or vendor rows become invisible to every user-JWT reader while still being fully accessible via the service role, an inconsistent, latent-bug posture.

1.8 No inbound FKs to the estimate tables (verified)

A scan of `supabase/migrations/` finds **no** `references estimate_config(...)` or `references estimate_matrix(...)` anywhere, only an index and a comment reference them. The `estimate_config` PK swap (§3.2) therefore breaks nothing. (Still confirm against the live DB before applying, in case of a dashboard-created constraint outside migrations.)

2. Key Decisions

2.1 Org-scoping: dual nullable FK + check, not owner_type/owner_id

Decision (per design review): add `vendor_id` uuid references `vendors(id)` to `workflow_steps`, `estimate_config`, `estimate_matrix`; drop NOT NULL on `studio_id`; add check `(num_nonnulls(studio_id, vendor_id) = 1)`; replace each `unique(studio_id, ...)` with **a single full unique index over a generated owner_key** (§2.2).

Why not the sync-layer owner_type/owner_id convention (used by `replicated_work`, `source_field_mappings`, `actuals_razor_config`): those tables have no FK to studios/vendors. The estimation tables already carry hard FKs to `studios(id)`, and three call sites filter on `studio_id`. Dual-FK (a) preserves DB-level referential integrity to **both** org types (security-first principle), (b) keeps every existing `studio_id` filter working with a **no-op backfill**, and (c) leaves the scenario reads (§1.6) untouched. The cost is two nullable columns + a check, and the NULL-distinct care in §2.2.

2.2 Uniqueness: a single owner_key full unique index (fork A, chosen)

The earlier draft proposed a hybrid, generated `owner_key` and per-owner partial indexes, which is incoherent: `owner_key = coalesce(studio_id, vendor_id)` under `num_nonnulls = 1` **already** fully enforces per-owner uniqueness, so the partials are redundant double-enforcement, and a conflicting insert could trip either index nondeterministically. The clean choice:

Sole enforcer and PostgREST arbiter = one full unique index over owner_key. No partial indexes.

```
-- owner_key is always non-null (the num_nonnulls=1 check guarantees exactly one owner is set)
owner_key uuid generated always as (coalesce(studio_id, vendor_id)) stored
```

- `estimate_config`: `unique (owner_key)`

- `workflow_steps`: `unique (owner_key, airtable_template_id)`, left **NULLS DISTINCT** (default) so multiple native steps (`airtable_template_id IS NULL`) coexist per owner, matching today's behaviour.
- `estimate_matrix`: `unique (owner_key, workflow_step_id, variable_values, link_id)` with **NULLS NOT DISTINCT** on the nullable `link_id` (PG15+) so two base rows (`link_id IS NULL`) collide. `owner_key`, `workflow_step_id`, `variable_values` are all non-null, so **NULLS NOT DISTINCT** affects only `link_id`, there is no separate "`link_key`" sentinel (that idea is dropped). *PG < 15 fallback*: split into base (`where link_id is null`) + override (`where link_id is not null`) unique indexes; the route then upserts against whichever applies.

`owner_key` doubles as the `on_conflict` arbiter for PostgREST upserts (single deterministic target), which resolves the original arbiter-inference problem. **Validate** that PostgREST `on_conflict=` binds to a **NULLS NOT DISTINCT** index during M-02 (it matches on the column list; the null-handling is an index property), if not, fall back to route-level select-then-write for the matrix-cell endpoint.

Note: `owner_key` is a bare uuid; a studio and a vendor sharing an identical uuid would be treated as the same owner. Under `gen_random_uuid()` this is not a practical concern. If belt-and-suspenders is ever wanted, redefine as text `('studio:' || studio_id) / ('vendor:' || vendor_id)`, not done now.

2.3 Per-link model: base matrix + overrides on estimate_matrix only

Add `link_id` uuid references `studio_vendor_links(id)` to `estimate_matrix`. Base rows: `link_id = NULL`. Override rows: `link_id` set, overriding `estimate_days` for a specific (`workflow_step_id`, `variable_values`). **Effective matrix for link L = base rows overlaid with L's override rows**, override winning on key collision; an override with no matching base simply contributes its value. A DB CHECK (§3.3) restricts `link_id` to vendor-owned rows.

`workflow_steps` and `estimate_config` get **no link_id**, process shape and variable axes are **vendor-global**; only rates diverge per link. (Standing assumption; if a vendor ever needs a genuinely different workflow *shape* per client, Phase 1 grows, not built now.)

2.4 Share = snapshot via a route-scoped inbox (RLS as defense-in-depth)

A share is a **frozen, self-describing** projection of the effective matrix at a granularity the vendor **picks per share** (the modal control), with **revoke, optional expiry**, and an **append-only access log**, exactly the post-token payload model (§1.4). Re-sharing to a link **replaces** the prior share regardless of granularity (§2.6), so the modal genuinely answers "how much to show *this time*," and a dial-back actually reduces what the studio can see.

Isolation boundary = route-layer owner filter, not RLS. Like the rest of the stack, the share endpoints use the **service-role client**, which bypasses RLS. The studio "inbox" is therefore a route-scoped read: `GET .../inbox` filters `recipient_studio_id = resolve_owner(studio)` and checks `revoked_at / expires_at` server-side. The owner filter on **every** share query is the enforced guarantee and must be airtight and unit-tested (§6.4). Calling this "RLS-inbox" would be a misnomer.

RLS on the new tables = defense-in-depth, added for parity with the existing estimate/payload tables (which all carry policies that only fire under a user JWT). The new share tables get JWT-claim

policies mirroring `payload_dispatches / payload_access_log` : vendor (sender) full access, studio (recipient) SELECT, and **no INSERT policy** on the log (service-role writes only). They will not fire at runtime under the service-role client, but keep the table set consistent if a user-JWT path is ever introduced.

Delivery is a pluggable strategy, not hardcoded. A `delivery_mode` column (default `'route_inbox'`, the only value now, named for what it is, §1.7, not the "RLS" misnomer) records the strategy; the dispatch route resolves a strategy object and calls `deliver(dispatch)`. The token path (external / non-ArtHound recipient, both directions, DocuSign-style) is a **reserved future strategy** that adds its own delivery-specific fields (e.g. `token_hash`) **without reshaping the snapshot/series/log core**. Not built now.

2.5 The asset-anchor exception (conscious, commented)

`estimate_share_dispatches` is deliberately **not** FK'd to `canonical_assets`, unlike `payload_dispatches`. An estimation matrix is a **rate card keyed on asset profiles (variable combos)**, a tier above any individual asset, with no asset linkage even today. This does not violate the "everything links to its canonical asset" rule (which governs asset/work/dispatch/review records). Captured as a SQL comment on the table so it reads as an intentional exception.

2.6 Share-series identity + auto-supersede on re-share (replace semantics)

A series is the **sharing channel for a relationship**, identified by `(vendor_id, link_id)`, **one channel per studio link**, enforced by `unique (vendor_id, link_id)`. **Granularity is a per-share property of the dispatch** (carried in the snapshot), *not* part of the channel key. This was a conscious fork:

- **(a) chosen, replace semantics.** Any re-share to a link supersedes the prior live dispatch in that one channel, *regardless of granularity*. A vendor who shared `workflow_step` and then re-shares `asset_total` to dial disclosure **down** genuinely reduces exposure, the detailed dispatch is superseded. One live share per relationship; §2.4's modal ("how much to show this time") is truthful.
- **(b) rejected, coexisting per-granularity channels.** Keying the series on granularity would let a dialed-back share land in a *different* channel while the higher-disclosure dispatch stays live, reintroducing the exact §6.5 IP leak one layer up, in the data model where the projector can't catch it. Simultaneous `asset_total` + `workflow_step` to the same studio is not a real use case; not worth the footgun.

On a new share the POST handler **finds-or-creates** the `(vendor, link)` series and **auto-supersedes** any existing live dispatch in it (`superseded_at = now()`, log `superseded`) before inserting the new one. **Active share** = the series' dispatch with `superseded_at IS NULL`, not revoked, not expired. At-most-one-live is made **structural** by a partial unique index (§3.5, §6.8), not left to the route layer, same DB-integrity stance as §2.1. The deferred user-facing REFRESH (explicit re-issue UX) is built on this same data path.

2.7 Explicit non-goals (do not build)

- **No semantic/basis discriminator on the estimate number** (no "internal forecast" vs "quoted price" type column). The number stays semantically open; per-org meaning. The snapshot carries `unit` (e.g. `days`) but no basis/type. If specificity is ever needed it is offered as user-defined paths at the *consuming* feature, never baked into this schema.
- **No billing semantics.** Coarse scenario-planning estimate boxes, not invoicing.
- **No user-facing REFRESH** (only the series identity + auto-supersede that precede it, §2.6).
- **No scenario-planner / baseline-promotion wiring.** The structured self-describing snapshot keeps it consumable later without a migration, don't pre-build the consumption.
- **No `estimate_share_field_mappings`** (Phase 4 vocabulary reconciliation) until a studio actually consumes the data.
- **No profile-subset sharing.** v1 shares all profiles in the effective matrix. If added later it is another per-share dispatch property under replace semantics (§2.6), not a channel key.

3. Schema

3.1 Migration sequence overview

Phase 0:		
M-01	<code>org-scope workflow_steps / estimate_config / estimate_matrix</code> (dual FK + owner_key full unique + extend RLS to vendor)	(expand – reversible)
Phase 1:		
M-02	<code>estimate_matrix.link_id + recreate owner_key unique w/ link_id</code> <code>+ link_id-vendor-only CHECK</code>	(additive – reversible)
Phase 2:		
M-03	<code>estimate_share_series</code> (granularity-keyed channel + unique)	(new table – reversible)
M-04	<code>estimate_share_dispatches</code>	(new table – reversible)
M-05	<code>estimate_share_access_log</code>	(new table – reversible)

All steps are additive/reversible. **No irreversible step**, `studio_id` is retained on the estimate tables (studio rows keep it; it is simply no longer mandatory), so no backfill-then-drop gate is needed.

3.2 M-01: org-scope the estimation stack (expand)

Run in one transaction. Look up the actual auto-generated unique-constraint name before dropping it (Postgres truncates long autonames at 63 chars), query `pg_constraint / \d estimate_matrix` rather than hardcoding `estimate_matrix_studio_id_workflow_step_id_variable_values_key`.

```

begin;

-- — estimate_matrix —————
alter table estimate_matrix add column if not exists vendor_id uuid references vendors(id);
alter table estimate_matrix alter column studio_id drop not null;
alter table estimate_matrix add constraint em_one_owner
  check (num_nonnulls(studio_id, vendor_id) = 1);
alter table estimate_matrix add column owner_key uuid
  generated always as (coalesce(studio_id, vendor_id)) stored;
-- drop the old single-owner unique (resolve real name first), replace with owner_key full u
-- link_id is added in M-02; the index is recreated there to include it.
alter table estimate_matrix drop constraint <resolved_old_unique_name>;
create unique index uq_em_owner on estimate_matrix (owner_key, workflow_step_id, variable_v

-- — estimate_config (PK swap - verified no inbound FKs, §1.8) —————
alter table estimate_config add column if not exists id uuid not null default gen_random_uu
alter table estimate_config add column if not exists vendor_id uuid references vendors(id);
alter table estimate_config alter column studio_id drop not null;
alter table estimate_config drop constraint estimate_config_pkey; -- was PRIMARY KEY (s
alter table estimate_config add constraint estimate_config_pkey primary key (id);
alter table estimate_config add constraint ec_one_owner check (num_nonnulls(studio_id, v
alter table estimate_config add column owner_key uuid
  generated always as (coalesce(studio_id, vendor_id)) stored;
create unique index uq_ec_owner on estimate_config (owner_key);

-- — workflow_steps —————
alter table workflow_steps add column if not exists vendor_id uuid references vendors(id);
alter table workflow_steps alter column studio_id drop not null;
alter table workflow_steps add constraint ws_one_owner check (num_nonnulls(studio_id, v
alter table workflow_steps add column owner_key uuid
  generated always as (coalesce(studio_id, vendor_id)) stored;
alter table workflow_steps drop constraint workflow_steps_studio_id_airtable_template_id_key
create unique index uq_ws_owner on workflow_steps (owner_key, airtable_template_id); -- NUL

-- — extend the existing studio-only RLS to vendor rows (§1.7) —————
-- Defense-in-depth only (routes use service role); keep vendor rows visible to user-JWT rec
create policy "ws_vendor" on workflow_steps for all using (
  vendor_id = (auth.jwt()→'app_metadata'→>'vendor_id')::uuid);
create policy "ec_vendor" on estimate_config for all using (
  vendor_id = (auth.jwt()→'app_metadata'→>'vendor_id')::uuid);
create policy "em_vendor" on estimate_matrix for all using (
  vendor_id = (auth.jwt()→'app_metadata'→>'vendor_id')::uuid);
-- dependencies: replace the studio-only FK-chain policy with an owner-aware one
drop policy if exists "wsd_studio" on workflow_step_dependencies;
create policy "wsd_owner" on workflow_step_dependencies for all using (
  step_id in (
    select id from workflow_steps
    where studio_id = (auth.jwt()→'app_metadata'→>'studio_id')::uuid
    or vendor_id = (auth.jwt()→'app_metadata'→>'vendor_id')::uuid
  ));

commit;

```

workflow_step_dependencies keeps its (step_id, depends_on_step_id) PK (ownership flows through the step). Backfill is a no-op, existing rows keep studio_id, and owner_key populates automatically. The pre-existing ws_studio / ec_studio / em_studio policies are left in place (permissive policies OR together: studio users match the studio policy, vendor users match the new vendor policy).

3.3 M-02: estimate_matrix.link_id + override uniqueness + CHECK

```
alter table estimate_matrix
  add column if not exists link_id uuid references studio_vendor_links(id);

-- link_id is meaningful only on vendor-owned override rows - enforce at the DB (§6.6)
alter table estimate_matrix add constraint em_link_vendor_only
  check (link_id is null or vendor_id is not null);

-- recreate the owner_key unique to include link_id; NULLS NOT DISTINCT so base rows (link_id is null)
drop index uq_em_owner;
create unique index uq_em_owner on estimate_matrix
  (owner_key, workflow_step_id, variable_values, link_id) nulls not distinct;

create index on estimate_matrix (link_id) where link_id is not null;
```

PG < 15 fallback: drop nulls not distinct and split into base (where link_id is null) + override (where link_id is not null) unique indexes; the route upserts against whichever applies (§2.2).

3.4 M-03: estimate_share_series (one channel per relationship)

```
create table estimate_share_series (
  id                uuid primary key default gen_random_uuid(),
  vendor_id         uuid not null references vendors(id),
  link_id           uuid not null references studio_vendor_links(id),
  recipient_studio_id uuid not null references studios(id), -- denormalized from link for t
  label            text,
  created_at        timestampz not null default now(),
  -- one channel per relationship; find-or-create + auto-supersede (§2.6).
  -- Granularity is a per-share dispatch property (in the snapshot), NOT a channel key - rep
  -- semantics, so a dial-back supersedes the prior detailed share rather than coexisting w
  unique (vendor_id, link_id)
);
create index on estimate_share_series (recipient_studio_id);

alter table estimate_share_series enable row level security; -- defense-in-depth (§2.4)
create policy "ess_vendor" on estimate_share_series for all using (
  vendor_id = (auth.jwt()→'app_metadata'→»'vendor_id')::uuid);
create policy "ess_studio_read" on estimate_share_series for select using (
  recipient_studio_id = (auth.jwt()→'app_metadata'→»'studio_id')::uuid);
```

3.5 M-04: estimate_share_dispatches

```
-- NOTE: deliberately NOT FK'd to canonical_assets. An estimation matrix is a rate card key
-- asset profiles (variable combos), a tier above any individual asset. Intentional, reviewed
-- exception to the canonical-asset linkage rule (see plan §2.5).
create table estimate_share_dispatches (
  id                uuid primary key default gen_random_uuid(),
  series_id         uuid not null references estimate_share_series(id),
  vendor_id         uuid not null references vendors(id),           -- sender (denormalized)
  recipient_studio_id uuid not null references studios(id),         -- denormalized for the
  link_id           uuid not null references studio_vendor_links(id),
  snapshot          jsonb not null,                                -- frozen self-describing
  delivery_mode      text not null default 'route_inbox'           -- widen when token str
                        check (delivery_mode in ('route_inbox')),   -- nullable: open-ended
  expires_at        timestamptz,                                   -- set on re-share (§2.
  revoked_at        timestamptz,
  superseded_at     timestamptz,
  created_at        timestamptz not null default now()
);
create index on estimate_share_dispatches (series_id);
create index on estimate_share_dispatches (recipient_studio_id);
create index on estimate_share_dispatches (vendor_id);

-- At-most-one LIVE dispatch per channel – structural, not route-layer (§2.6, §6.8).
-- A concurrent double-share makes the losing insert fail this unique → the handler retries.
-- Expiry is intentionally excluded: it is time-based (not indexable), and an expired share
-- an accretion duplicate.
create unique index uq_esd_one_live on estimate_share_dispatches (series_id)
  where superseded_at is null and revoked_at is null;

alter table estimate_share_dispatches enable row level security; -- defense-in-depth (§2.4)
create policy "esd_vendor" on estimate_share_dispatches for all using (
  vendor_id = (auth.jwt()→'app_metadata'→»'vendor_id')::uuid);
create policy "esd_studio_read" on estimate_share_dispatches for select using (
  recipient_studio_id = (auth.jwt()→'app_metadata'→»'studio_id')::uuid
  and revoked_at is null
  and (expires_at is null or expires_at > now()));
```

Granularity is a **per-share property carried in the snapshot**, neither the series nor the dispatch has a granularity column (single source of truth; §2.6). The inbox reads it from the snapshot and may embed `estimate_share_series(label)`. `superseded_at` filtering ("show only the active share") is applied in the query, not RLS, so history remains readable if a future UI wants it.

Snapshot shape (self-describing, forward-compatible):

```
{
  "schema_version": 1,
  "vendor": { "id": "...", "handle": "..." },
  "link_id": "...",
  "granularity": "craft_bucket",
  "variable_fields": ["Item Type", "Complexity"],
  "unit": "days",
  "generated_at": "2026-05-29T...Z",
  "profiles": [
    {
      "variable_values": { "Item Type": "Axe", "Complexity": "High" },
      "total_days": 30,
      "breakdown": [
        { "label": "Modeling", "days": 12 },
        { "label": "Texturing", "days": 18 }
      ]
    }
  ]
}
```

- `asset_total` → per-profile `total_days` only; **no** `breakdown` (no step or craft detail leaks).
- `craft_bucket` → `breakdown` labels are **crafts**, steps summed within each craft.
- `workflow_step` → `breakdown` labels are **step names** (the vendor's explicit choice to expose process detail at share time).

The projector must emit only what the chosen granularity permits, it is the IP-exposure boundary.

3.6 M-05: estimate_share_access_log (append-only)

```
create table estimate_share_access_log (
  id                uuid primary key default gen_random_uuid(),
  dispatch_id       uuid not null references estimate_share_dispatches(id) on delete cascade,
  event             text not null,    -- 'shared' | 'viewed' | 'revoked' | 'superseded'
  actor_vendor_id   uuid references vendors(id),
  actor_studio_id   uuid references studios(id),
  detail            jsonb,
  created_at        timestampz not null default now()
);
create index on estimate_share_access_log (dispatch_id);
create index on estimate_share_access_log (created_at desc);

alter table estimate_share_access_log enable row level security; -- defense-in-depth (§2.4)
-- SELECT for both parties via the dispatch FK chain; NO INSERT policy (service-role _log())
-- mirroring payload_access_log exactly.
create policy "esal_vendor_select" on estimate_share_access_log for select using (
  dispatch_id in (select id from estimate_share_dispatches
    where vendor_id = (auth.jwt()→'app_metadata'→>'vendor_id')::uuid));
create policy "esal_studio_select" on estimate_share_access_log for select using (
  dispatch_id in (select id from estimate_share_dispatches
    where recipient_studio_id = (auth.jwt()→'app_metadata'→>'studio_id')::uuid);
```

Append-only, never updated or deleted, mirroring `payload_access_log`.

4. Code Changes

4.1 New: owner-resolution shim (Phase 0)

Add to `lib/auth.py` (or a small `lib/owner.py`):

```
def resolve_owner(user: CurrentUser) → tuple[str, str]:
    """Return (owner_col, owner_id) for the estimate stack: ('studio_id'|'vendor_id', uuid).
    if user.role == "studio" and user.studio_id:
        return "studio_id", user.studio_id
    if user.role == "vendor" and user.vendor_id:
        return "vendor_id", user.vendor_id
    raise HTTPException(status_code=403, detail="No organisation linked to this account")
```

This `(owner_col, owner_id)` pair is the **isolation boundary** (§2.4/§6.4): every estimate-stack and share query must filter on it. Treat a missing filter as a tenancy bug.

4.2 routes/matrix.py + routes/workflow_steps.py, owner-aware (Phase 0)

- Switch dependencies from `require_studio` to `get_current_user` + `resolve_owner`.
- Replace `studio_id=eq.{studio_id}` filters / insert payloads with the resolved `(owner_col, owner_id)`.
- Change upserts to the **owner_key arbiter** (§2.2): `on_conflict=owner_key` for `estimate_config`, `on_conflict=owner_key, workflow_step_id, variable_values, link_id` for `estimate_matrix`. The two call sites to update are `create_matrix_pg` (`on_conflict=studio_id` and the matrix-rows upsert) and `update_matrix_cell`. (Insert payloads still set `studio_id` / `vendor_id`; `owner_key` is generated.)
- `randomize-matrix` stays admin-only via `require_admin(current_user)` (unchanged logic, owner-aware filter).
- Behaviour for studio callers is byte-for-byte unchanged (studio rows keep `studio_id`).

4.3 Per-link override layer (Phase 1)

- `GET /api/setup/matrix-table-pg?linkId=<uuid>` (vendor): build the **effective** matrix, base rows overlaid with that link's override rows, and tag each cell `{ value, source: 'base'|'override' }`. Without `linkId`, returns the base matrix (existing behaviour for the owner).
- `PATCH /api/setup/matrix-cell` (vendor): accept optional `link_id`. With `link_id`, upsert an **override** row (`link_id` set); without, write the **base** row. Validate the link belongs to the calling vendor (`require_active_link`); the `em_link_vendor_only` CHECK (§3.3) is the DB backstop.
- New `lib/estimate/effective.py`: `resolve_effective_matrix(vendor_id, link_id)`, the single source of truth for `base⊕override` resolution, reused by both the editor view and the projector.

4.4 Projection + delivery (Phase 2)

- New `lib/estimate/projector.py`: `project(effective_matrix, steps, config, granularity) → snapshot`. Sums the step axis per §3.5; enforces the granularity exposure boundary (no breakdown

for `asset_total` ; craft labels for `craft_bucket` ; step names only for `workflow_step`).

- New `lib/estimate/delivery.py` : `DeliveryStrategy` protocol + `RlsInboxDelivery` (the only impl). `get_delivery_strategy('route_inbox')` returns it; the dispatch route calls `strategy.deliver( ... )`. Token strategy is a documented stub, not implemented.
- New `routes/estimate_share.py` :
- **Vendor** (`require_vendor`):
 - `GET /api/estimate-shares/targets` , active studio links available to share to.
 - `POST /api/estimate-shares` , { `link_id` , `granularity` , `expires_in_days?` , `label?` } : validate active link → build effective matrix → project (granularity into the snapshot) → **find-or-create** the (`vendor` , `link`) series → **auto-supersede** any existing live dispatch in it (`superseded_at = now()` , log `superseded`) → insert the new dispatch (via strategy) → log `shared` (§2.6). Do the supersede + insert in **one transaction** (a Postgres RPC, since PostgREST requests are individually transactional) so there is no transient no-live window; the `uq_esd_one_live` index (§3.5) is the structural backstop, on a concurrent double-share the losing insert hits the unique and the handler retries the flow. *Replace semantics: a re-share at any granularity supersedes the prior, including a dial-down (§2.6).*
 - `GET /api/estimate-shares/outbox` , vendor's series with their active dispatch + state.
 - `POST /api/estimate-shares/{id}/revoke` , set `revoked_at` , log `revoked` .
- **Studio** (`require_studio`):
 - `GET /api/estimate-shares/inbox` , active dispatches where `recipient_studio_id = studio` (route-scoped filter; `superseded_at` is null , not revoked/expired), returning the snapshot.
 - `POST /api/estimate-shares/{id}/view` , log `viewed` (optional telemetry).
- Reuse the payload patterns: a local `_log( ... )` writing `estimate_share_access_log` , and an `_assert_valid( ... )` raising **410** on revoked/expired, mirroring [routes/payload.py](#).

4.5 Frontend

- **Phase 0**: no new screens, the existing OrgHub Estimates/Workflows tabs (§1.1) become functional for vendors once the endpoints are owner-aware. Verify `api.js` `apiFetch` calls work under a vendor session.
- **Phase 1**: in the vendor's Estimates tab, add a studio-link selector; render the effective matrix with base values shown and an editable override indicator (`source: 'override'` highlighted).
- **Phase 2**: a "Share estimates" action on the vendor's studio-connection view (`/studios` → `StudioConnections.jsx` , *confirm component*): modal to pick granularity + expiry, preview the projection, confirm. An outbox list with revoke.
- **Phase 3**: on the studio's vendor-connection view (`/vendors` → `VendorConnections.jsx` , *confirm component*): a received-estimates panel rendering the snapshot **read-only** by granularity (totals table for `asset_total` ; grouped rows for `craft_bucket` ; per-step for `workflow_step`). No scenario wiring.

4.6 Unchanged

- `lib/scenario/shared.py`, `lib/scenario/context.py`, `routes/scenario.py`, untouched; their `studio_id` filters keep returning studio-owned rows only (§1.6).
- `lib/sync/*`, the share feature does not touch the sync pipeline.
- `workflow_step_dependencies` schema and handlers, unchanged (only its RLS policy is broadened, §3.2).

5. Phase Split

Phase	Scope	Migrations	Independently shippable?
**0,	0, Org-scope	Vendors author their own base matrix + workflow via the existing OrgHub UI	M-01
**1,	1, Per-link overrides	Effective-matrix resolver + per-studio override editing	M-02
**2,	2, Projection + share	Projector, delivery strategy seam, series identity + auto-supersede, vendor share/outbox/revoke API + UI	M-03, M-04, M-05
**3,	3, Studio viewer	Studio inbox + read-only snapshot viewer	,

Phase 0 is the prerequisite and the largest single risk surface (touches shared matrix routes + RLS); everything after is additive.

6. Open Risks

6.1 PostgREST on_conflict against a NULLS NOT DISTINCT index

The `owner_key` full unique index is the single arbiter (§2.2). The one thing to validate during M-02: PostgREST `on_conflict=owner_key,workflow_step_id,variable_values,link_id` must bind to a `NULLS NOT DISTINCT` index (it matches on the column list; null-handling is an index property). If it doesn't, fall back to route-level select-then-write for the matrix-cell upsert. Not a schema fork, the index stays; only the write path changes.

6.2 PG version for NULLS NOT DISTINCT

`link_id` -nullable uniqueness (§3.3) relies on PG15 `NULLS NOT DISTINCT`. Confirm the Supabase Postgres major version; if < 15, use the base/override split-index fallback. (Dev and prod are separate Supabase projects, verify both.)

6.3 Vocabulary mismatch defers consumption

A vendor's `variable_fields` / profile values need not match the studio's. v1 is visible-only, so the studio sees the **vendor's own labels**, acceptable. Genuine consumption (scenario planning) requires reconciliation (the deferred `estimate_share_field_mappings`, §2.7). Do not let "visible" imply "usable" in UI copy.

6.4 Route-layer owner filter is the isolation boundary (RLS is bypassed)

Resolved posture (§1.7, §2.4): the estimate and share tables carry RLS policies, but every route uses the **service-role client**, which **bypasses RLS**, so RLS only fires for a user-JWT PostgREST reader, which the app never is. The enforced runtime boundary is the **route-layer owner filter** on every read and write; a missing filter crosses the studio/vendor boundary. M-01 extends the existing studio-only policies to vendor rows so the *defense-in-depth* layer stays consistent (it does not change runtime behaviour).

Action: test the route filters as the boundary, a cross-tenant request (studio A's JWT, studio B's share id) must return empty/403 at the route, not rely on RLS.

6.5 Granularity exposure is the IP boundary

The projector is the only thing between a vendor's internal process detail and the studio. A bug that emits `breakdown` for `asset_total`, or step names under `craft_bucket`, leaks IP the vendor chose not to share. Cover the projector with unit tests asserting the exposure rule per granularity (no DB needed, pure function over the effective matrix). The **data model** must not undermine this: under replace semantics (§2.6) a dial-back supersedes the prior higher-disclosure share, so a vendor reducing granularity actually reduces what the studio can read, the leak that a granularity-keyed channel would have reintroduced is closed.

6.6 link_id-vendor-only is enforced in the DB

`estimate_matrix.link_id` is meaningful only on vendor-owned override rows. Enforced by the `em_link_vendor_only` CHECK (§3.3), `check (link_id is null or vendor_id is not null)`, consistent with the §2.1 choice of DB-level integrity for the dual-FK. The route also validates the link belongs to the calling vendor (`require_active_link`); the CHECK is the structural backstop.

6.7 Share of an empty / partial matrix

A vendor may share before fully populating overrides. The projector reads the effective matrix as-is; cells with no value contribute 0 (consistent with the base matrix's default-0 seeding in `create_matrix_pg`). Surface "n cells unset" in the share-preview UI so a vendor doesn't ship zeros unknowingly.

6.8 Concurrent re-share race

"One live dispatch per channel" (§2.6) is not safe if left to the route layer: a double-clicked share or two concurrent vendor users could each supersede the same prior dispatch and each insert, leaving two live. The `uq_esd_one_live` partial unique index (§3.5), `(series_id)` where `superseded_at is null` and `revoked_at is null`, makes at-most-one-live **structural**: the losing racer fails the unique and the

handler retries. Run supersede + insert in one transaction/RPC (§4.4) to avoid a transient no-live window. Expiry is deliberately not in the index (time-based, not indexable; an expired share is not an accretion duplicate).

7. Sequenced Milestones

M0, Org-scope the stack (Phase 0). M-01 (owner_key full unique + dual-FK + extend RLS to vendor) + `resolve_owner` shim + owner-aware [matrix.py](#) and [workflow_steps.py](#). Verify studio behaviour is unchanged, a vendor session can create a matrix via OrgHub, and a cross-tenant request returns empty at the route (§6.4). Reviewable: one migration + auth shim + two route diffs + a tenancy test.

M1, Per-link overrides (Phase 1). M-02 (link_id + recreate owner_key unique w/ link_id + `em_link_vendor_only` CHECK) + `lib/estimate/effective.py` + `linkId` on the matrix read/cell-write endpoints + vendor override editing UI. Reviewable: migration + effective-resolver unit tests + route diffs + UI.

M2, Projection + share API (Phase 2 backend). M-03/M-04/M-05 + `lib/estimate/projector.py` (with the §6.5 exposure tests) + `lib/estimate/delivery.py` (RlsInbox only) + `routes/estimate_share.py` (vendor create-with-auto-supersede/outbox/revoke, studio inbox/view). Reviewable: migrations + projector tests + new route module + an auto-supersede test (re-share leaves exactly one live dispatch per channel).

M3, Vendor share UI (Phase 2 frontend). "Share estimates" modal (granularity, expiry, preview) + outbox + revoke on the vendor's studio-connection view.

M4, Studio viewer (Phase 3). Studio received-estimates panel rendering the snapshot read-only by granularity. Completes v1.

Deferred beyond this plan: user-facing REFRESH, `estimate_share_field_mappings` vocabulary reconciliation, scenario-planner consumption / baseline promotion, profile-subset sharing, and the token delivery strategy for external recipients.